8051 Assembly Programming

Module 2

- Introduction to 8051 assembly programming, Data types and Assembler directives, 8051 Addressing Modes, simple Assembly language programs(data transfer and arithmetic operations only)
- 8051 programming in C: Data types and time delay in 8051, 8051 Timer/Counter programming in embedded C: Programming 8051 timers, Counter programming,

Assembly Language

- The 8051 Microcontroller Assembly Language is a combination of English like words called Mnemonics and Hexadecimal codes.
- It is also a low level language and requires extensive understanding of the architecture of the Microcontroller.

Why Assembly Language?

- The Programs written in Assembly gets executed faster and they occupy less memory.
- With the help of Assembly Language, you can directly exploit all the features of a Microcontroller.
- Using Assembly Language, you can have direct and accurate control of all the Microcontroller's resources like I/O Ports, RAM, SFRs, etc.

Structure of the 8051 Assembly Language

- Each line or statement of the assembly language program of 8051 Microcontroller consists of three fields:
- Label, Instruction and Comments.

Instruction Structure



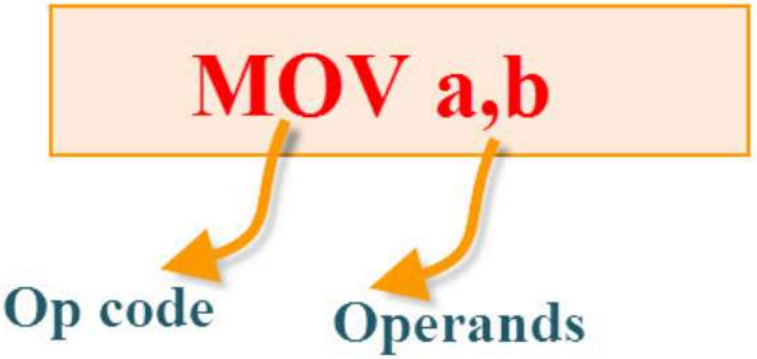
Fig: 8051 Instruction Structure

Instruction Structure

- **Label :**
- The Label is programmer chosen name for a Memory Location or a statement in a program.
- **Instruction :**
- The Instruction is the main part of the 8051 Microcontroller Assembly Language Programming as it is responsible for the task performed by the Microcontroller.
- Any Instruction in the Assembly Language consists of two parts: **Op-code and Operand(s).**

Instruction Structure





Instruction Structure

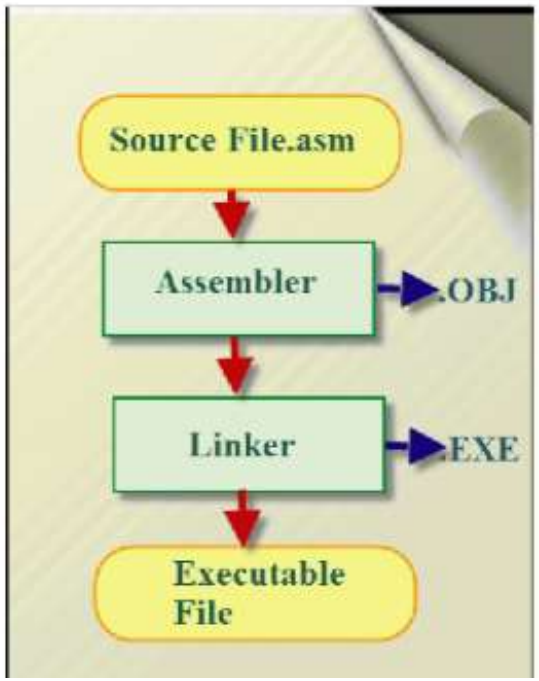
- **Op-code** The op-code is a single instruction that can be executed by the CPU. Here the op-code is a MOV instruction.
- **Operands** The operands are a single piece of data that can be operated by the op-code.
- Example, multiplication operation is performed by the operands that are multiplied by the operand.

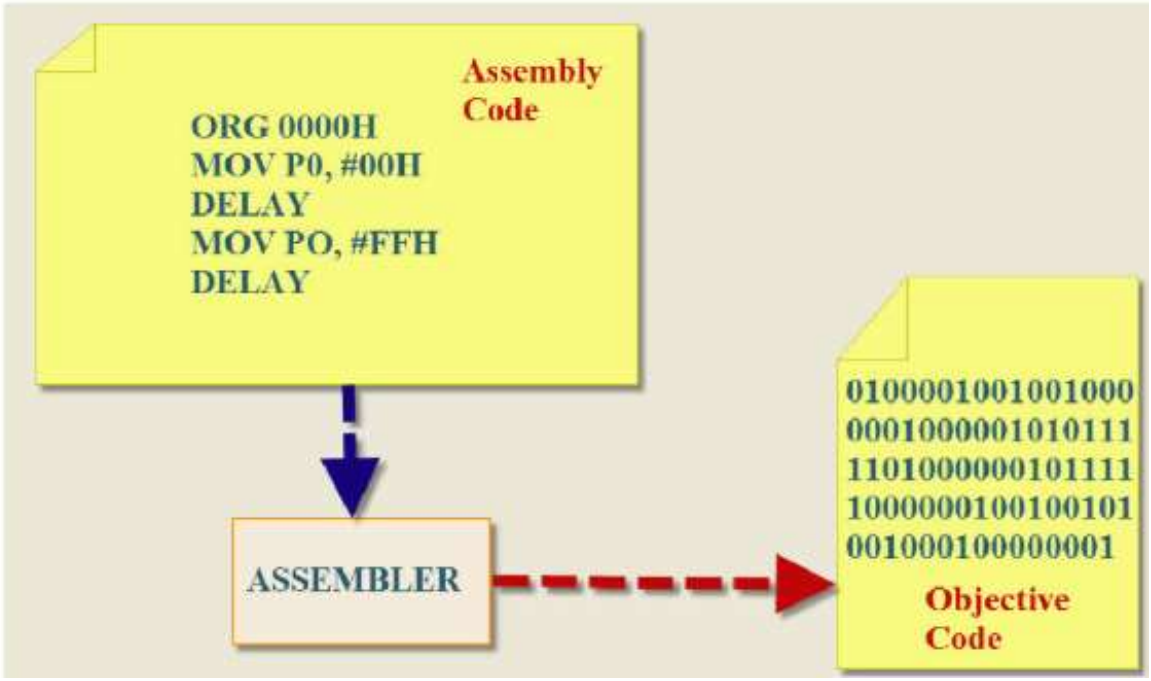
  Syntax: **MUL a,b;**

Comments

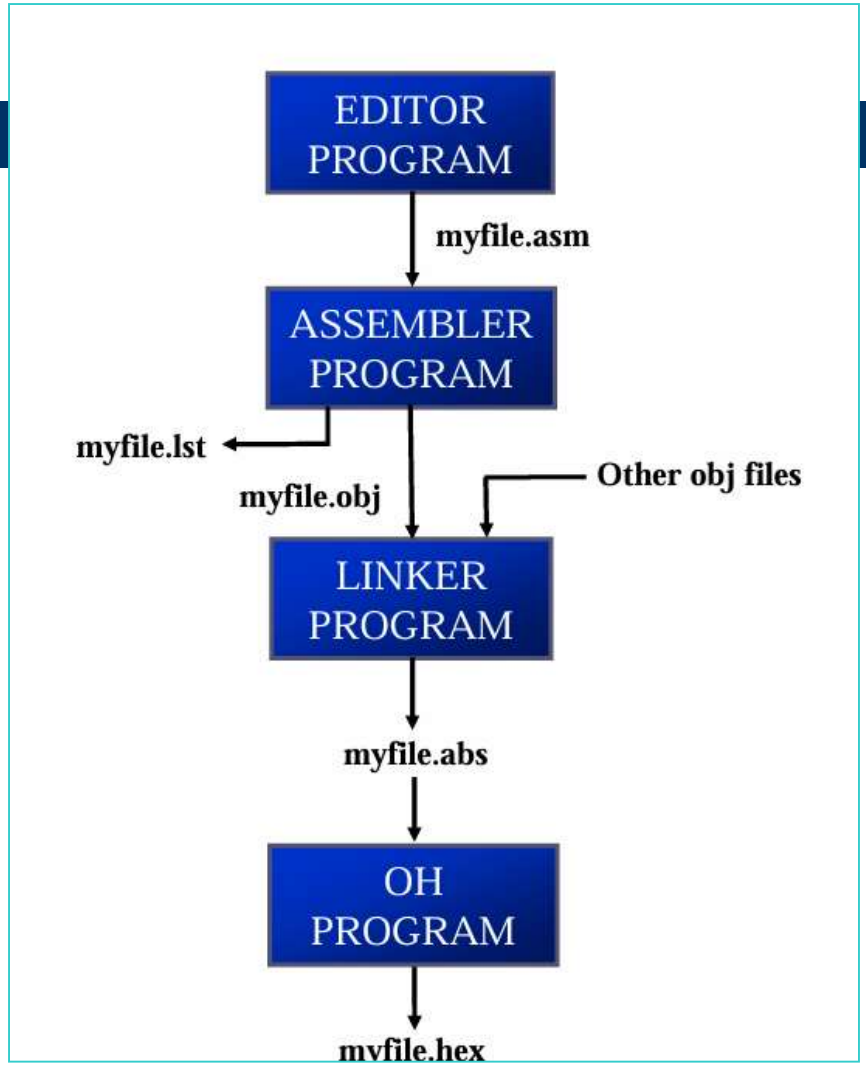
- **Comments :**
- The last part of the Structure of 8051 Assembly Language is the Comments. Comments are statements included by the developer for easier understanding of the code and is used for proper documentation of the Program.

Assembling and Running an 8051 Program





Assembling and Running an 8051 Program



Assembling and Running an 8051 Program

- The step of Assembly language program are outlines as follows:
- 1) First we use an editor to type a program, many excellent editors or word processors are available that can be used to create and/or edit the program
- The source file has the extension “asm“or “src”, depending on which assembly you are using

Assembling and Running an 8051 Program

- **2**) The “asm”source file containing the program code created in step 1 is fed to an 8051 assembler
  - The assembler converts the instructions into machine code
    - The assembler will produce an object file and a list file
    - The extension for the object file is “obj”while the extension for the list file is “lst
    - [The lst (list) file, which is optional, is very useful to the programmer lst File HANEL It lists all the opcodes and addresses as well as errors that the assembler detected The programmer uses the lst file to find the syntax errors or debug]

Assembling and Running an 8051 Program

- 3) Assembler require a third step called linking
- The linker program takes one or more object code files and produce an absolute object file with the extension “abs”
- This abs file is used by 8051 trainers that have a monitor program

Assembling and Running an 8051 Program

- 4) Next the “abs”file is fed into a program called “OH”(object to hex converter) which creates a file with extension “hex” that is ready to burn into ROM
- This program comes with all 8051 assemblers
- Recent Windows-based assemblers combine step 2 through 4 into one step

Assembling and Running an 8051 Program

- **Program the Microcontroller (Hardware Implementation)**
  - Transfer the HEX file into the 8051 microcontroller using a device programmer.
  - Connect the microcontroller to the target circuit.
  - Power the circuit and verify the program operation.

Assembling and Running an 8051 Program

- **Program the Microcontroller (Hardware Implementation)**
  - Transfer the HEX file into the 8051 microcontroller using a device programmer.
  - Connect the microcontroller to the target circuit.
  - Power the circuit and verify the program operation.

Elements of an Assembly Language Programming

- Assembler Directives
- Instruction Set
- Addressing Modes

8051 Microcontroller Assembly Directives

- Assembly Language Directives are actually instructions to the Assembler and directs the Assembler Program what to do during the process of Assembling.
  - **ORG (origin)**
  - **EQU**
  - **DB**
  - **END**

ORG (origin)

- **ORG (origin)**
- used to specify the **starting memory address** where the assembler should place the following program or data.
- It is **not an executable instruction**,
- The number that comes after ORG can be either in hex or in decimal.
- Eg: **ORG** 0000H

EQU

- **EQU** (Equate)
  Using the EQU Directive, you can associate a Symbol (or Label) with a Value.
- TMP **EQU** #30 ; Assigns the value #30 to the name TMP
- RED_LED **EQU** P1.0 ; P1.0 is defined as RED_LED

DB (DEFINE BYTE)

- The DB directive is used to define an 8 bit data.
- TABLE: DB 12H, 34H, 56H, 78H
- MESSAGE: DB 'HELLO'

END

- END
- The END directive signals the end of the assembly module. It indicates the end of the program to the assembler.
- Any text in the assembly file that appears after the END directive is ignored.
- If the END statement is missing, the assembler will generate an error message

MICROCONTROLLER INSTRUCTION SET

Data Type

- The 8051 microcontroller contains a single data type of 8-bits, and each register is also of 8-bits size.
- The programmer has to break down data larger than 8-bits (00 to FFH, or to 255 in decimal) so that it can be processed by the CPU.

MICROCONTROLLER INSTRUCTION SET

Introduction

- An instruction is an order or command given to a processor by a computer program.
- All commands are known as instruction set and set of instructions is known as program.
- 8051 have in total **111** instructions, i.e. 111 different words available for program writing.

Instruction Format

- Where first part describes **WHAT** should be done, while other explains **HOW** to do it.
- The latter part can be a **data (binary number) or the address at which the data is stored**.
- Depending upon the number of bytes required to represent 1 instruction completely.

Types Of Instructions

- Instructions are divided into 3 types;

1. One/single byte instruction.-In 1-byte instruction, the opcode and the operand of an instruction are represented in one byte
2. Two/double byte instruction.
3. Three/triple byte instruction.

Types Of Instructions

1. One/single byte instructions :

- If operand is not given in the instruction or there is no digits present with instruction, the instructions can be completely represented in one byte opcode.

- OPCODE 8 bit

Types Of Instructions

2. Two/double byte instruction:

- If 8 bit number is given as operand in the instruction, the such instructions can be completed represented in two bytes.

- First byte OPCODE
- Second byte 8 bit data or I/O port

Types Of Instructions

3. Three/triple byte instruction:

- If 16 bit number is given as operand in the instructions than such instructions can be completely represented in three bytes 16 bit number specified may be data or address.

Types Of Instructions

1. First byte will be instruction code.
2. Second byte will be 8 LSB's of 16 bit number.
3. Third byte will be 8 MSB's of 16 bit number.

- First byte OPCODE.
- Second byte 8 LSB's of data/address.
- Third byte 8 MSB'S of data/address.

Addressing Modes

- The way by which **address of data** is given in **an instruction** is termed as Addressing mode
- Addressing modes specifies where the data (operand) is. They specify the source or destination of data (operand) in several different ways, depending upon the situation.
- Addressing modes are used to know where the operand located is.

Addressing Modes of 8051

- There are 5 types of addressing modes:

1. Register addressing.
2. Direct addressing.
3. Register indirect addressing.
4. Immediate addressing.
5. Index addressing.

Register Addressing Mode

- In register addressing mode; the source and/or destination is a register.

- In this case; data is placed in any of the 8 registers(R0-R7); in instructions it is specified with letter Rn (where N indicates 0 to 7).

Register Addressing Mode

- For example;

1. ADD A, Rn (This is general instruction).

2. ADD A, R5 (This instruction will add the contents of register R5 with the accumulator contents).

Direct Addressing Mode

- In direct addressing mode; the address of memory location containing data to be read is specified in instruction.

- In this case; address of the data is given with the instruction itself.

Direct Addressing Mode

- For example;

1. MOV A, **25H** (This instruction will read/move the data from internal RAM address 25H and store it in the accumulator.

Register Indirect Addressing Mode

- In register indirect addressing mode; the contents of the designated register are used as a pointer to memory.

- In this case; data is placed in memory, but address of memory location is not given directly with instruction.

Register Indirect Addressing Mode

- For example;

1. MOV A,@R0 This instruction moves the data from the register whose address is in the R0 register into the accumulator.

Immediate Addressing Mode

- In immediate addressing mode, the data is given with the instruction itself.

- In this case; the data to be stored in memory immediately follows the opcode.

Immediate Addressing Mode

- For example;

1. MOV A, #25H (This instruction will move the data 25H to accumulator.

Index Addressing Mode

- Offset (from accumulator) is added to the base index register(DPTR OR Program Counter) to form the effective address of the memory location.

- In this case; this mode is made for reading tables in the program memory.

Index Addressing Mode

- For example;

1. MOVC A, @ A + DPTR (This instruction moves the data from the memory to accumulator; whose address is computed by adding the contents of accumulator and DPTR)

Types Of Instructions

1. Data transfer instructions.
2. Arithmetic instructions.
3. Logical instructions.
4. Bit level Instructions
5. Logical instructions with bits
6. Branch instructions.

Types Of Instructions

DATA TRANSFER	*ARITHMETIC*	*LOGICAL*	*BOOLEAN*	*PROGRAM BRANCHING*
MOV	ADD	ANL	CLR	LJMP
MOVC	ADDC	ORL	SETB	AJMP
MOVX	SUBB	XRL	MOV	SJMP
PUSH	INC	CLR	JC	JZ
POP	DEC	CPL	JNC	JNZ
XCH	MUL	RL	JB	CJNE
XCHD	DIV	RLC	JNB	DJNZ
	DA A	RR	JBC	NOP
		RRC	ANL	LCALL
		SWAP	ORL	ACALL
			CPL	RET
				RETI
				JMP

Types of Instructions

8051 INSTRUCTION TYPES (1/2)

Instruction Type	Purpose	Examples
1. Data Transfer Instructions 	Transfer data between registers, memory, and I/O ports.	MOV, MOVC, MOVX, PUSH, POP, XCH, XCHD
2. Arithmetic Instructions 	Perform mathematical operations such as addition, subtraction, multiplication, and division.	ADD, ADDC, SUBB, INC, DEC, MUL AB, DIV AB, DA A
3. Logical Instructions 	Perform logical operations like AND, OR, XOR, complement, rotate, swap, and clear.	ANL, ORL, XRL, CLR, CPL, RL, RLC, RR, RRC, SWAP

Types of Instructions

8051 INSTRUCTION TYPES (2/2)

Instruction Type	Purpose	Examples
4. Branch (Program Control) Instructions 	Change the sequence of program execution based on conditions or unconditionally.	SJMP, AJMP, LJMP, JZ, JNZ, JC, JNC, DJNZ, CJNE, ACALL, LCALL, RET, RETI, NOP
5. Bit Manipulation Instructions 	Operate on individual bits in RAM or Special Function Registers (SFRs).	SETB, CLR, CPL, JB, JNB, JBC, MOV C,bit, MOV bit,C, ANL C,bit, ORL C,bit

Data Transfer Instructions

- These instructions move the content of one register to another one.

- Data can be transferred to stack with the help of PUSH and POP instructions.

Types of Instructions

Mnemonic	Description
MOV	Move Data
MOVC	Move Code
MOCX	Move External Data
PUSH	Move Data to Stack
POP	Copy Data from Stack
XCH	Exchange Data between two Registers
XCHD	Exchange Lower Order Data between two Registers

Data Transfer Instructions

MNEMONIC	DESCRIPTION	BYTES
MOV A,Rn	(A) ← (Rn)	1
MOV A,@Ri	(A) ← (Ri)	1

Data Transfer Instructions

- MOV A,#X (A) ← Data 2
- MOV Rn,A (Rn)← (A) 1
- MOV Rn, Rx (Rn)← (Rx) 2

Data Transfer Instructions

- MOV Rn, #X (Rn) ← Data 2
- MOV Rx, A (Rx) ← (A) 2
- MOV Rx, Rn (Rx) ← (Rn) 2

Data Transfer Instructions

- MOV Rx, Ry (RX) ← (Ry) 3
- MOV Rx, @ Ri (Rx) ← (Ri) 2
- MOV Rx, # X (Rx) ← Data 3

Data Transfer Instructions

- MOV @ Ri, A (Ri)← (A) 1
- MOV @ Ri, Rx (Ri) ← (Rx) 2
- MOV @ Ri, #X (Ri) ← Data 2

Data Transfer Instructions

- MOV DPTR, #X (DPTR) ←Data 3
- MOVC A @ A+DPTR (A) ←(A+DPTR) 1
- MOVC A@ A+PC (A)←(A+PC) 1

Data Transfer Instructions

- MOVX A,@ Ri A ← (Ri) 1
- MOVX A, @ DPTR (A) ← (DPTR) 1
- MOVX @Ri, A (Ri) ← (A) 1

Data Transfer Instructions

- MOVX @ DPTR, A — (DPTR)← (A) — 1
- PUSH Rx — Push directly addressed Rx register on stack — 2
- POP Rx — (A) ← (Rx) — 2

Data Transfer Instructions

- XCH A, Rn (A)↔(Rn) 1
- XCH A, @Ri (A)↔ (Ri) 1

↔

Data Transfer Instructions

- XCHD Exchange 4 lower bits in accumulator with indirectly addressed register 1

Arithmetic Instructions

- These instructions perform several basic operations. After execution, the result is stored in the first operand.

- 8 bit addition, subtraction, multiplication, increment-decrement instructions can be performed.

Arithmetic Instructions

Mnemonic	Description
ADD	Addition without Carry
ADDC	Addition with Carry
SUBB	Subtract with Carry
INC	Increment by 1
DEC	Decrement by 1
MUL	Multiply
DIV	Divide
DA A	Decimal Adjust the Accumulator (A Register)

Arithmetic Instructions

MNEMONICS	DESCRIPTION	BYTE
ADD A, Rn	A = A + Rn	1
ADD A, Rx	A = A + Rx	2
AAD A, @ Ri	A = A+ Ri	1

Arithmetic Instructions

- ADD A, # X A = A + Byte 2
- ADDC A, Rn A = A + Rn + C 1
- ADDC A , Rx A = A + Rx + C 2

Arithmetic Instructions

• ADDC A, @ Ri	A = A + Ri + C	1
• ADDC A, # X	A = A + Byte + C	2
• SUBB A, Rn	A = A – Rn – 1	1

Arithmetic Instructions

• SUBB A, Rx	A = A – Rx – 1	2
• SUBB A, @ Ri	A = A – Ri – 1	1
• SUBB A, # X	A = A – Byte – 1	2

Arithmetic Instructions

• INC A	A = A + 1	1
• INC Rn	Rn = Rn + 1	1
• INC Rx	Rx = Rx + 1	2

Arithmetic Instructions

• INC @ Ri	Ri = Ri + 1	1
• DEC A	A = A – 1	1
• DEC Rn	Rn = Rn – 1	1

Arithmetic Instructions

• DEC Rx	Rx = Rx – 1	2
• DEC @ Ri	Ri = Ri – 1	1
• INC DPTR	DPTR = DPTR + 1	1

Arithmetic Instructions

- MUL AB B:A = A * B 1
- DIV AB A = [A/B] 1
- DA A Decimal adjustment of accumulator according to BCD code 1

Logical Instructions

- These instructions perform logical operations between two register contents on bit by bit basis.

- After execution, the result is stored in the first operand.

Logical Instructions

Mnemonics	Description	Bytes
ANL A, Rr	A AND Rr-->A	1
ANL A, ADD	A AND(ADD)-->A	2
ORL A, Rr	A OR Rr-->A	1
ORL A, ADD	A OR(ADD)-->A	2
XRL A, Rr	A XOR Rr-->A	1
XRL A, ADD	A XOR(ADD)-->A	2
CLR A	00-->A	1
CPL A	A bar-->A	2

Logical Instructions

Mnemonic	Description
ANL	Logical AND
ORL	Logical OR
XRL	Ex-OR
CLR	Clear Register
CPL	Complement the Register
RL	Rotate a Byte to Left
RLC	Rotate a Byte and Carry Bit to Left
RR	Rotate a Byte to Right
RRC	Rotate a Byte and Carry Bit to Right
SWAP	Exchange lower and higher nibbles in a Byte

Logical Instructions

MNEMONIC	DESCRIPTION	BYTE
ANL A, Rn	(A)← (A) ^ (Rn)	1
ANL A, Rx	(A)← (A) ^ (Rx)	2
ANL A,@ Ri	(A)← (A) ^ (Ri)	1

Logical Instructions

- ANL A, # X (A)← (8 bit data) ^ (A) 2
- ANL Rx, A (Rx) (A) ^ (Rx) 2
- ANL Rx,# X (Rx)← (8 bit data) ^ (Rx) 3

Logical Instructions

- ORL A, Rn (A) ← (A) + (Rn) 1
- ORL A, Rx (A) ← (A) + (Rx) 2
- ORL A, @ Ri (A) ← (A) + (Ri) 2

Logical Instructions

- ORL Rx, A (Rx) ← (A) + (Rx) 2
- ORL Rx,# X (Rx) ← (8 bit data) + (Rx) 2
- XORL A, Rn Logical exclusive 1
  OR operation between the contents of accumulator and R register.

Logical Instructions

- XORL A, Rx Logical exclusive OR 2 operation between the contents of the accumulator and directly addressed register Rx.
- XORL A,@ Ri Logical exclusive OR 1 operation between the contents of the accumulator and directly addressed register.

Logical Instructions

- XORL A, # X Logical exclusive OR 2 operation between the contents of accumulator and the given 8 bit data.
- XORL Rx, A Logical exclusive OR 2 operation between the contents of the accumulator and directly addressed register Rx.

Logical Instructions

- XORL Rx, # X Logical exclusive OR operation between the contents of the directly addressed register Rx and the given 8 bit data. 3
- CLR A (A)← 0 1
- CPL A (A) ← (/A) 1

Logical Instructions

- SWAP A (A3-0) ⟷ (A7-4) 1
- RL A (An + 1) ← (An) 1
  (A0) ← (A7)
- RLC (An + 1) ← (An) 1
  (A0) ← (C)
  (C) ← (A7)

Logical Instructions

- RR A
  (An) ← (An + 1)
  (A7) ← (A0)
  1

- RRC A
  (An) ← (An + 1)
  (A7) ← (C)
  (C) ← (A0)
  1

Bit Level Instructions

- Similar to logical instructions, these instructions also perform logical operations.

- The difference is that these operations are performed on single bits.

Bit Level Instructions

- Instructions that are used for signal-bit operations are as following

Single-Bit Instructions

Instruction		Function
SETB	bit	Set the bit (bit = 1)
CLR	bit	Clear the bit (bit = 0)
CPL	bit	Complement the bit (bit = NOT bit)
JB	bit, target	Jump to target if bit = 1 (jump if bit)
JNB	bit, target	Jump to target if bit = 0 (jump if no bit)
JBC	bit, target	Jump to target if bit = 1, clear bit (jump if bit, then clear)

Bit Level Instructions

Mnemonic		Description	Byte	Cycle
Boolean Variable Manipulation				
CLR	C	Clear carry flag	1	1
CLR	bit	Clear direct bit	2	1
SETB	C	Set carry flag	1	1
SETB	bit	Set direct bit	2	1
CPL	C	Complement carry flag	1	1
CPL	bit	Complement direct bit	2	1
ANL	C,bit	AND direct bit to carry flag	2	2
ANL	C,/bit	AND complement of direct bit to carry	2	2
ORL	C,bit	OR direct bit to carry flag	2	2
ORL	C,/bit	OR complement of direct bit to carry	2	2
MOV	C,bit	Move direct bit to carry flag	2	1
MOV	bit,C	Move carry flag to direct bit	2	2

Bit Level Instructions

Mnemonic	Description
CLR	Clear a Bit (Reset to 0)
SETB	Set a Bit (Set to 1)
MOV	Move a Bit
JC	Jump if Carry Flag is Set
JNC	Jump if Carry Flag is Not Set
JB	Jump if specified Bit is Set
JNB	Jump if specified Bit is Not Set
JBC	Jump if specified Bit is Set and also clear the Bit
ANL	Bitwise AND
ORL	Bitwise OR
CPL	Complement the Bit

Bit Level Instructions

Mnemonic	Instruction	Description	# of Bytes	# of Cycles
CLR	C	C ← 0 (C = Carry Bit)	1	1
	Bit	Bit ← 0 (Bit = Direct Bit)	2	1
SET	C	C ← 1	1	1
	Bit	Bit ← 1	2	1
CPL	C	C ← $\overline{C}$	1	1
	Bit	Bit ← $\overline{Bit}$	2	1
ANL	C, /Bit	C ← C. $\overline{Bit}$ (AND)	2	1
	C, Bit	C ← C . Bit (AND)	2	1
ORL	C, /Bit	C ← C + $\overline{Bit}$ (OR)	2	1
	C, Bit	C ← C + Bit (OR)	2	1
MOV	C, Bit	C ← Bit	2	1
	Bit, C	Bit ← C	2	2
JC	rel	Jump is Carry (C) is Set	2	2
JNC	rel	Jump is Carry (C) is Not Set	2	2
JB	Bit, rel	Jump is Direct Bit is Set	3	2
JNB	Bit, rel	Jump is Direct Bit is Not Set	3	2
JBC	Bit, rel	Jump is Direct Bit is Set and Clear Bit	3	2

ELECTRONICS HUB

Logical Instructions On Bits

MNEMONIC	DESCRIPTION	BYTE
CLR C	(C = 0)	1
CLR bit	clear directly addressed bit	2
SETB C	(C = 1)	1

Logical Instructions On Bits

• SETB bit addressed bit	Set directly	2
• CPL C	(1 = 0, 0 = 1)	1
• CPL bit addressed bit	Complement directly	2

Logical Instructions On Bits

- ANL C, bit Logical AND operation 2 between Carry bit and directly addressed bit.

- ANL C,/bit Logical AND operation 2 between Carry bit and inverted directly addressed bit.

Logical Instructions On Bits

- ORL C, bit Logical OR operation 2
  between Carry bit and directly addressed bit.

- ORL C,/bit Logical OR operation 2
  between Carry bit and inverted directly addressed bit.

Logical Instructions On Bits

- MOV C, bit — Move directly addressed bit to carry bit. — 2
- MOV bit, C — Move Carry bit to directly addressed bit. — 2

Program Flow Control Instructions

- In this group, instructions are related to the flow of the program, these are used to control the operation like, **JUMP and CALL** instructions.

- Some instructions are used to introduce delay in the program, to the halt program.

Program Flow Control Instructions

Mnemonic	Description
LJMP	Long Jump (Unconditional)
AJMP	Absolute Jump (Unconditional)
SJMP	Short Jump (Unconditional)
JZ	Jump if A is equal to 0
JNZ	Jump if A is not equal to 0
CJNE	Compare and Jump if Not Equal
DJNZ	Decrement and Jump if Not Zero
NOP	No Operation
LCALL	Long Call to Subroutine
ACALL	Absolute Call to Subroutine (Unconditional)
RET	Return from Subroutine
RETI	Return from Interrupt
JMP	Jump to an Address (Unconditional)

Pr...ns

Mnemonic	Instruction	Description	# of Bytes	# of Cycles
ACALL	ADDR11	Absolute Subroutine Call PC + 2 → (SP); ADDR11 → PC	2	2
LCALL	ADDR16	Long Subroutine Call PC + 3 → (SP); ADDR16 → PC	3	2
RET	--	Return from Subroutine (SP) → PC	1	2
RETI	--	Return from Interrupt	1	2
AJMP	ADDR11	Absolute Jump ADDR11 → PC	2	2
LJMP	ADDR16	Long Jump ADDR16 → PC	3	2
SJMP	rel	Short Jump PC + 2 + rel → PC	2	2
JMP	@A + DPTR	A + DPTR → PC	1	2
JZ	rel	If A=0, Jump to PC + rel	2	2
JNZ	rel	If A ≠ 0, Jump to PC + rel		
CJNE	A, Direct, rel	Compare (Direct) with A. Jump to PC + rel if not equal	3	2
	A, #Data, rel	Compare #Data with A. Jump to PC + rel if not equal	3	2
	Rn, #Data, rel	Compare #Data with Rn. Jump to PC + rel if not equal	3	2
	@Ri, #Data, rel	Compare #Data with @Ri. Jump to PC + rel if not equal	3	2
DJNZ	Rn, rel	Decrement Rn. Jump to PC + rel if not zero	2	2
	Direct, rel	Decrement (Direct). Jump to PC + rel if not zero	3	2
NOP		No Operation	1	1

ELECTRONICS HUB

Program Flow Control Instructions

MNEMONIC	DESCRIPTION	BYTE
ACALL adr11	(PC) ← (PC) + 2	2
	(SP) ← (SP) + 1	
	((SP)) ← (PC7 – 0) -low order PC	
	(SP) ← (SP) + 1	
	((SP)) ← (PC15-8)- high order PC	

ACALL

ACALL addr11

Function:	Absolute Call within 2K block
Description:	ACALL unconditionally calls a subroutine located at the indicated address. The instruction increments the PC twice to obtain the address of the following instruction, then pushes the 16-bit result onto the stack (low-order byte first) and increments the Stack Pointer twice. The destination address is obtained by successively concatenating the five high-order bits of the incremented PC, opcode bits 7-5, and the second byte of the instruction. The subroutine called must therefore start within the same 2 K block of the program memory as the first byte of the instruction following ACALL. No flags are affected.
Example:	Initially SP equals 07H. The labs " SUBRTN " is at program memory location 0345 H. After executing the instruction, ACALL SUBRTN at location 0123H, SP will contain 09H, internal RAM locations 08H and 09H will contain 25H and 01H, respectively, and the PC will contain 0345H.
Bytes:	2
Clocks:	24
Encoding:	a10 a9 a8 1:0 0 0 1 a7 a6 a5 a4:a3 a2 a1 a0
Operation:	(PC) « (PC) + 2 (SP) « (SP) + 1 [(SP)] « (PC7-0) (SP) « (SP) + 1 [(SP)] « (PC15-8) (PC10-0) « page address
Flags:	None

Program Flow Control Instructions

- LCALL adr16

(PC) ← (PC) + 3	3
(SP) ← (SP) + 1	
((SP)) ← (PC7-0)	
(SP) ← (SP) + 1	
((SP)) ← (PC15-8)	
(PC) ← addr15-0	

Program Flow Control Instructions

- RET (PC15-8) ⟵ ((SP)) 1
  (SP) ⟵ (SP) – 1
  (PC7-0) ⟵ ((SP))
  (SP) ⟵ (SP) - 1

Program Flow Control Instructions

- RET1 (PC15-8) ⟵ ((SP)) 1
  (SP) ⟵ (SP) – 1
  (PC7-0) ⟵ ((SP))
  (SP) ⟵ (SP) – 1

- AJMP addr11 (PC) ⟵ (PC) + 2 1
  (PC10-0) ⟵ page address

Program Flow Control Instructions

- LJMP addr16 (PC)← addr15-0 3
- SJMP rel short jump from 2
  (from -128 to +127 locations in
  relation to first next instruction)

Program Flow Control Instructions

- JC rel (PC) ⟵ (PC) + 2 2

  IF (C) = 1

  THEN (PC) ⟵ (PC) + rel

- JNC rel (PC) ⟵ (PC) + 2 2

  IF (C) = 0

  THEN (PC) ⟵ (PC) + rel

Program Flow Control Instructions

- JB bit, rel — Jump if addressed bit is set. Short jump. — 3
- JBC bit, rel — Jump if addressed bit is set and clear it. Short jump. — 3

Program Flow Control Instructions

- JMP @A + DPTR (PC)$\leftarrow$ (A) + (DPTR) 1

- JZ rel (PC)$\leftarrow$ (PC) + 2 2

  IF (A) = 0

  THEN (PC)$\leftarrow$ (PC) + rel

Program Flow Control Instructions

- JNZ rel (PC) ← (PC) + 2 2
  IF (A) ≠ 0
  THEN (PC) ← (PC) + rel

- CJNE A, Rx, rel Compare the contents 3
  of acc. And directly addressed register Rx.
  Jump if they are different. Short jump.

Program Flow Control Instructions

- CJNE A, #X, rel (PC) ⟵ (PC) + 3 3

  IF (A) < > data

  THEN (PC) ⟵ (PC) + relative offset

  IF (A) < data

  THEN (C) ⟵ 1

  ELSE (C) ⟵ 0

Program Flow Control Instructions

- CJNE @ RI, # x, rel (PC) ⟵ (PC) + 3 3

  IF (Rn) <> data

  THEN (PC) ⟵ (PC) + relative offset

  IF (Rn) < data

  THEN (C) ⟵ 1

  ELSE (C) ⟵ 0

Program Flow Control Instructions

- CJNE @ Ri, # X, rel (PC)⟵(PC) + 3 3

  IF ((Ri)) <> data

  THEN (PC)⟵ (PC) + relative offset

  IF ((Ri)) < data

  THEN (C) ⟵ 1

  ELSE (C)⟵ 0

Program Flow Control Instructions

- DJNZ Rn , rel

(PC) ← (PC) + 2 2

(Rn) ← (Rn) - 1

IF (Rn) > 0 or (Rn) < 0

THEN (PC) ← (PC) + rel

Program Flow Control Instructions

- DJNZ Rx, rel

  (PC)⟵ (PC) + 2

  (Rx) ⟵ (Rn) – 1

  IF (Rx) > 0 or (Rx) < 0

  THEN (PC)⟵ (PC) + rel

  3

- NOP

  No operation

  1

Summary

- Instruction set.
- Addressing modes.
- Data transfer instruction.
- Arithmetic instruction.
- Logical instruction.
- Logical operation on bits.